

情報システム工学実験

オブジェクト指向プログラミング

(4) stream

令和8年5月18日(月)

情報・経営システム工学専攻 吉田

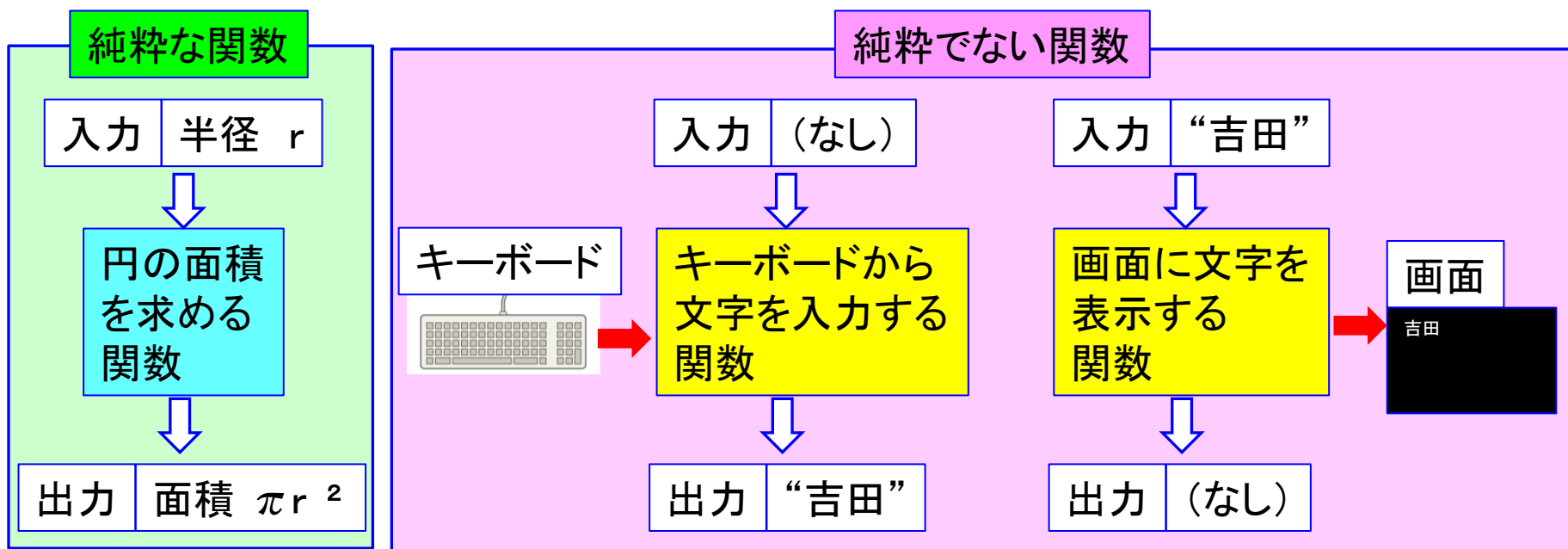


関数型プログラミング

関数には純粋な関数と純粋でない関数がある

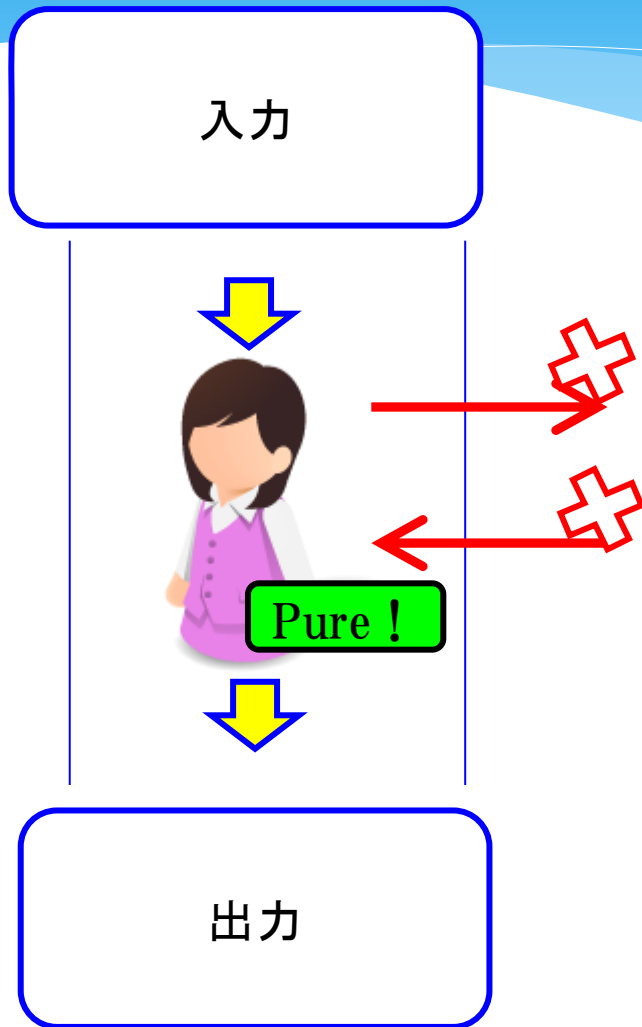
実は「関数」は、「純粋な関数」と「純粋でない関数」に分けられます。

「純粋な関数」は、出力以外に影響を与えませんし、何度実行しても、入力が同じなら、必ず同じ出力が得られます。



しかし「純粋でない関数」は、出力以外に影響を与えたり、入力が同じでも、実行するたびに(その日の気分で?)出力が変わってしまいます。

純粋な関数とは？



(1) 関数は、出力にのみ影響を与える

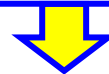
(2) 出力は、入力にのみ依存する

(3) 同じ入力を与えれば、
必ず同じ出力が得られる。

純粹じゃない人たち ①



を **5部** コピーしておいて。



SNS

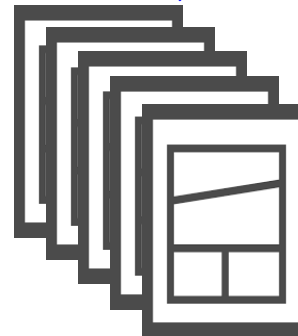
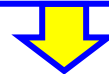
社外秘の書類
初めて見た！



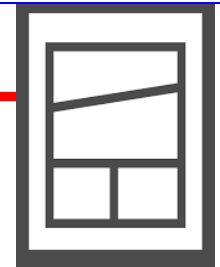
純粹じゃない人たち ②



を **5部** コピーしておいて。



ワン・ソース

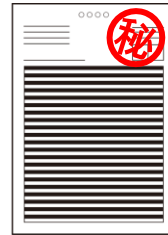


純粹じゃない人たち ③



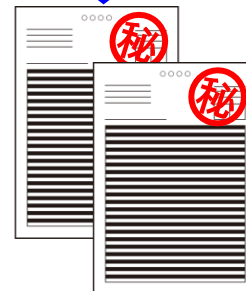
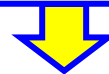
1回目

を 5部 コピーして。

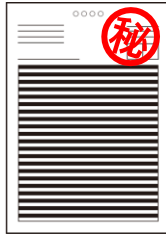


2回目

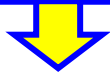
を 5部 コピーして。



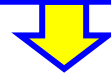
(仕事としては問題はあるかもしれないが) 純粋な人たち



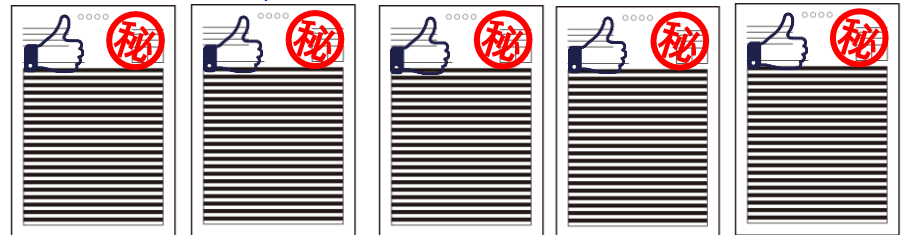
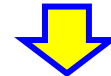
を **5部** コピーしておいて。



Pure !

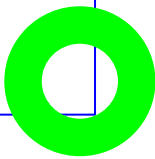


Pure !

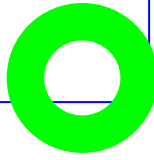


純粋な関数の例

```
public int multi(int a, int b){  
    return a * b;  
}
```



```
public boolean isLeapYear(int year){  
    if( year % 400 == 0 ){  
        return true;  
    }else if( year % 100 ==0 ){  
        return false;  
    }else if( year % 4 == 0 ){  
        return true;  
    }  
    return false;  
}
```



(1)関数は、出力にのみ影響を与える

(2)出力は、入力にのみ依存する

(3)同じ入力を与えれば、
必ず同じ出力が得られる。

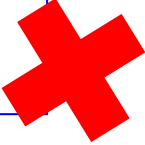
純粹ではない関数の例

関数内部で(勝手に)入力してる！

(同じ入力でも)実行するたびに結果が変わる！

入力
関数

```
public int input(){  
    Scanner sc = new Scanner(System.in);  
    return sc.nextInt();  
}
```



関数内部で(勝手に)出力してる！

出力
関数

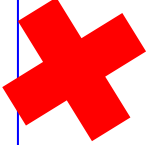
```
public void output(String msg){  
    System.out.println( msg );  
}
```



(同じ入力でも)実行するたびに結果が変わる！

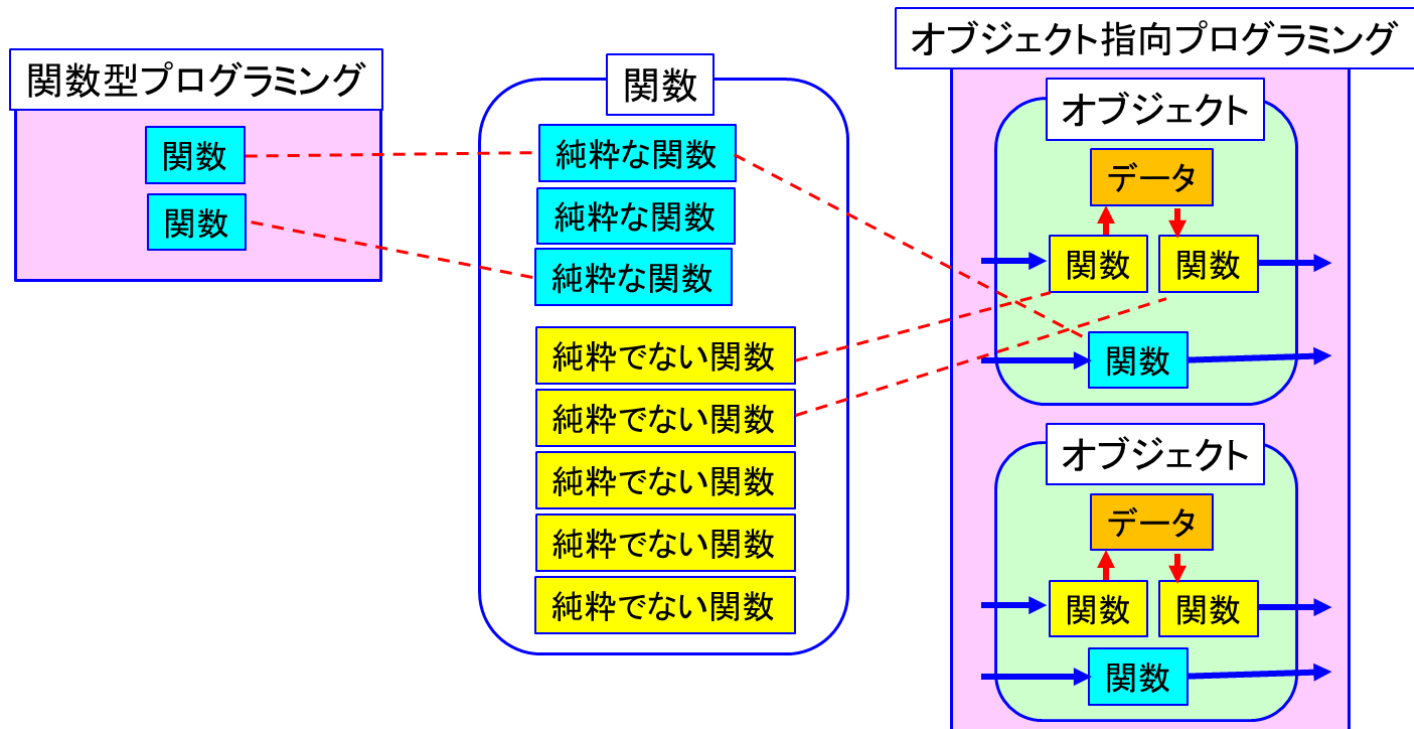
状態
関数

```
int count;  
public int counter( ){  
    count = count +1  
    return count;  
}
```



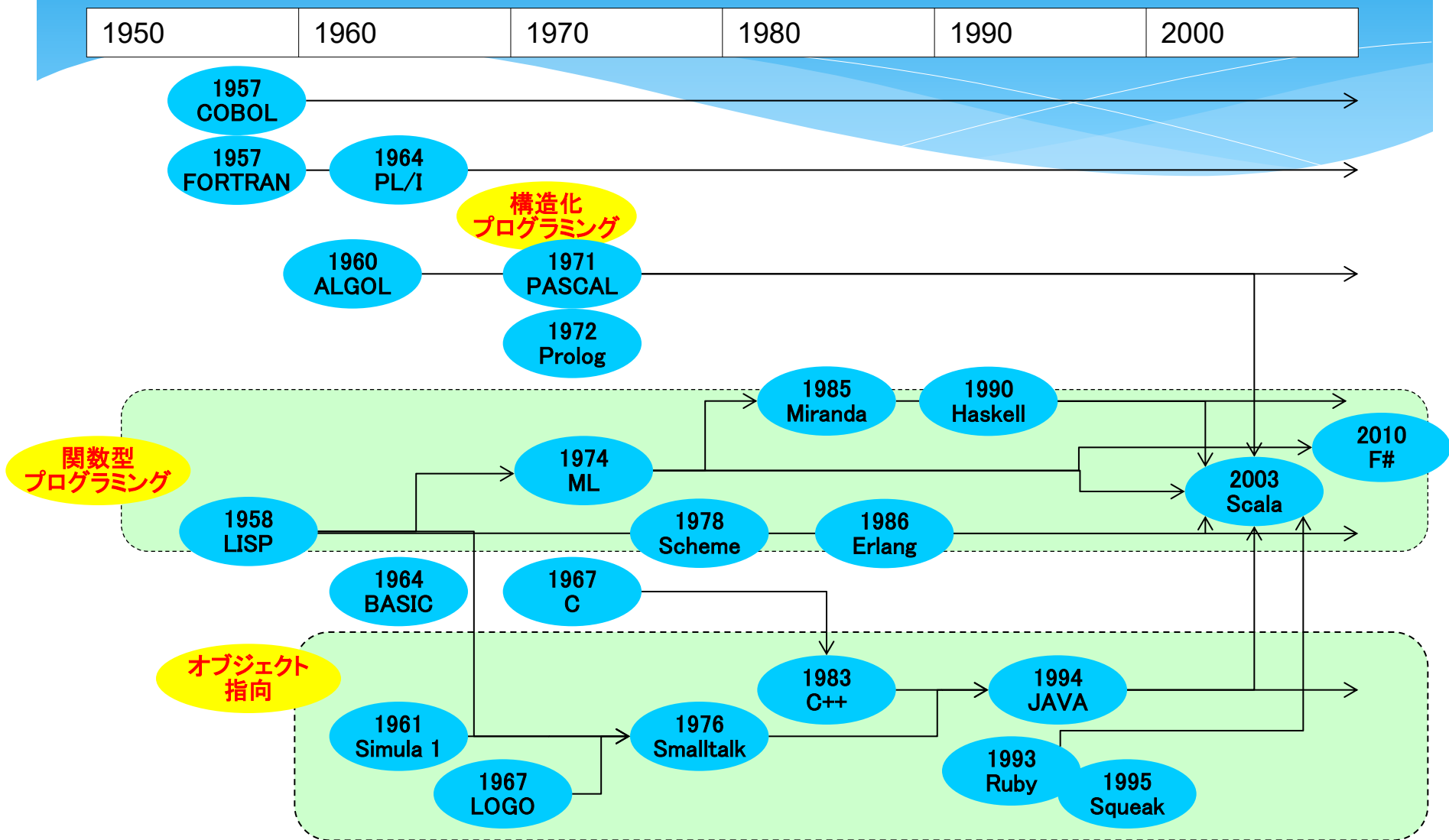
関数型プログラミング

「オブジェクト指向プログラミング」が幅を利かせていた時代にも
「純粋な関数」だけですべてのプログラムを作成できないだろうか？
と考えてきた人たちがいました。



彼らはこれを「関数型プログラミング」と呼んでいました。

プログラミング言語の進化図



純粋な関数のメリット① プログラムしやすい

入力



Pure !



出力



(1) 関数は、出力にのみ影響を与える

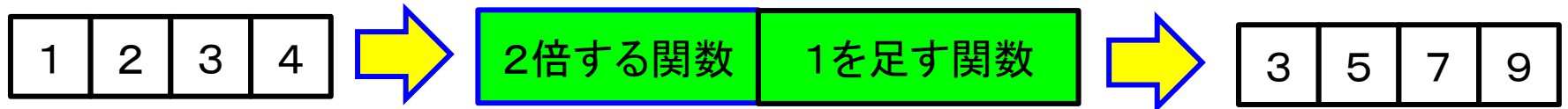
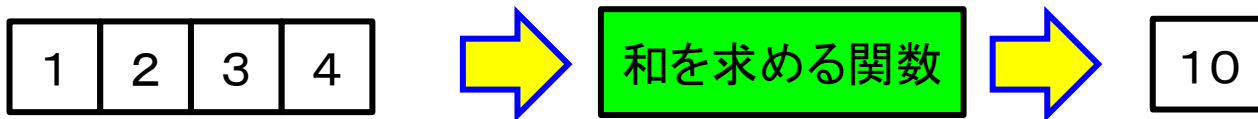
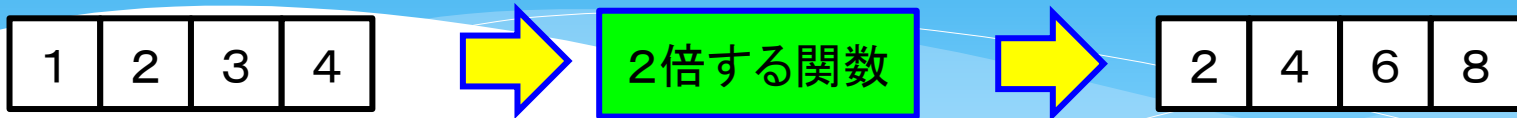
(2) 出力は、入力にのみ依存する

(3) 同じ入力を与えれば、
必ず同じ出力が得られる。

純粋な関数は、

- ・外部の変数の状態を考慮する必要がない。
- ・入力から出力を求めることだけを考えれば良い。

純粋な関数のメリット② 接続できる



純粋な関数をつなげるだけでプログラムになる

関数型プログラミング

関数型プログラミング

入力 → **Pure !**
関数 → 出力

入力 → **Pure !**
関数 → 出力

入力 → **Pure !**
関数 → 出力

入力 → **Pure !**
関数 → 出力

関数型プログラムの世界 (Before)



わが国民が皆
純粋だったら良
いのになあ

関数型プログラミング王国

あいつらだけは
どうしようもない！

あいつらが純粋
なはずがない！

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !



入力関数

出力関数

状態関数

関数型プログラムの世界 (After)



わが国民は
やはり皆、純粹
であった！

関数型プログラミング王国

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

関数

Pure !

Pure !

出力関数



入力関数

Pure !

状態関数

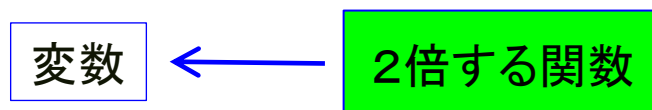
Pure !

モナド界

関数型プログラミング言語の特徴

(1) 関数を、変数に代入できます。

整数や文字列と同じように、関数も変数に代入したり加工したりできます。



(2) 変数の値は、変更できません。

変数は、中間の結果を入れておく場所なので変更する必要がありません。



(3) for文は、ありません。

For文を使う代わりに、リスト(≒配列)をまるごと変換します。

通常のパログラミングと関数型プログラミング

例) 1~10までの和を求めるプログラム

通常のパログラミング

「処理の内容」
を記述している



手続き的な
プログラミング

```
public static void main(String[] args) throws IOException {  
    int answer=0;  
    for( int i=0; i< 10; i++ ){  
        answer += i;  
    }  
    System.out.println( answer );  
}
```

関数型プログラミング

「処理の意味」
を記述している



宣言的な
プログラミング

```
public static void main(String[] args) throws IOException {  
  
    int answer = IntStream.rangeClosed(1,10).sum( );  
  
    System.out.println( answer );  
}
```

ラムダ式

関数とラムダ式

関数

2つの整数を加算する関数

```
public int add( int n , int m ){  
    return n + m ;  
}
```

ラムダ式

2つの整数を加算するラムダ式

```
( int n , int m ) -> { return n+m; } ;
```

「ラムダ式」は「関数」と同じ！

ラムダ式の例

2つの整数の最小値を求める関数

```
public static int isGreaterThan( int num1 , int num2 ) {  
    if (num1 < num2 ){  
        return num1;  
    } else {  
        return num2;  
    }  
}
```

ラムダ式

```
( int num1 , int num2 ) -> {  
    if (num1 < num2 ){  
        return num1;  
    } else {  
        return num2;  
    }  
};
```

月の日数を求めるラムダ式(=関数)

```
public static int getDayOfMonth( int month ) {  
    if(month==4 || month==6 || month==9||month=11){  
        return 30;  
    }else if(month==2){  
        return 28;  
    }else{  
        return 30;  
    }  
}
```

ラムダ式

```
( int month ) -> {  
    if(month==4 || month==6 || month==9||month=11){  
        return 30;  
    }else if(month==2){  
        return 28;  
    }else{  
        return 30;  
    }  
};
```

Streamを利用したプログラミング

Streamを使った基本パターン

生成操作
(複数の値を生成)

中間操作
(複数の値を抽出・変換)

終端操作
(複数の値から結果を生成)

例) 1から10までの整数を生成して、偶数のみを抽出し、その和を求める。

例)

```
IntStream.rangeClosed(1,10).filter( n -> n%2==0 ).sum( );
```

生成操作

中間操作

終端
操作

通常プログラミングと関数型プログラミング

例) 1~10までの偶数の和を求めるプログラム

通常プログラミング

「処理の内容」
を記述している



手続き的な
プログラミング

```
public static void main(String[] args) throws IOException {  
    int answer=0;  
    for( int i=0; i< 10; i++ ){  
        if( i % 2 ==0 ){  
            answer += i;  
        }  
    }  
    System.out.println( answer );  
}
```

関数型プログラミング

「処理の意味」
を記述している



宣言的な
プログラミング

```
public static void main(String[] args) throws IOException {  
  
    int answer = IntStream.rangeClosed(1,10)  
        .filter(n -> n%2==0 )  
        .sum( );  
  
    System.out.println( answer );  
}
```

Stream処理の基本操作

生成 操作	<code>intStream.range(from , to)</code>	fromからto-1までの整数を生成
	<code>intStream.rangeClosed(from , to)</code>	fromからtoまでの整数を生成
	<code>List<Card> cards; cards.stream()</code>	リストをストリームに変換する
中間 操作	<code>filter(IntPredicate)</code>	条件に一意した要素を抽出
	<code>map(IntUnaryOperator)</code>	要素を別の値に変換
終端 操作	<code>sum()</code>	全要素の和を求める
	<code>count()</code>	全要素の数を求める
	<code>anyMatch(IntPredicate)</code>	条件に一致する要素があるか調べる
	<code>allMatch(IntPredicate)</code>	全要素が条件に一致するか調べる
	<code>findFirst().getAsInt()</code>	条件に一致する最初の要素を1つ取り出す



オブジェクト指向プログラミング と 関数型プログラミング

オブジェクト指向プログラミングと 関数型プログラミングの癒合

というわけで「関数型プログラミング」が注目されるようになったのですが、「オブジェクト指向プログラミング」はすでに世界中の多くのプログラムに使われていて、それらを全部一から作り直すのは現実的ではありませんでした。

そこで「オブジェクト指向プログラミング」の人達は妥協案を見つけました。

「オブジェクト指向プログラミング」の言語に、「関数型プログラミング」の仕組みを少しだけ取り入れたのです。

「モナド」という考え方を取り入れることはできませんでしたが、「純粋な関数」を扱いやすくしたのです。

現在では、多くの「オブジェクト指向プログラミング」の言語で、「純粋な関数」を扱う仕組みが提供されるようになってきています。